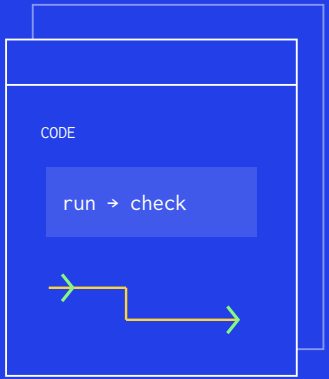
WEEK 3 / LAB 5 / TEAM WORK

MLOps: Machine Learning Operations

Run it.
Prove what ran.

Run your Lab 4 service in a container, make it log every request, and check from outside which model and code answer. Needs Lab 4 merged on main (or develop).



By the end you can show three things

Each one must be visible in your team repository.

1 **Run it**

Start your Lab 4 service from its Docker image, with settings you pass in, using only the steps in your README.

2 **Log it**

Every request writes one log line. Someone who reads only the logs can tell what happened.

3 **Check it**

A script outside the container asks which model and code are running, and fails if they are not the ones you expect.

Who does what in your team

The split is yours; every member owns something and reviews something.

EACH MEMBER

Own one part

Split it yourselves: start-up, logging, checks or README. One PR each.

THE WHOLE TEAM

Define “deployed”

Write in the README what “deployed” means, and who runs the final check.

REVIEWER

Review a PR

Take a PR you did not write. Change how it starts; check the evidence.

Individual report reports/week-3.md covers Labs 5–6 in your personal repository.

What your team must add

Everything goes in through reviewed pull requests.

REQUIRED

Open pull requests into your integration branch and have them reviewed. If you add a library, declare it in pyproject.toml and update uv.lock.

TEAM REPOSITORY PATHS

```
Dockerfile    extend the Lab 4 one
README.md     add a “Deploy” section
team report   add a “Lab 5” section
```

The README must let a stranger start and check the service.

Three tools you will use

Lectures 4 and 5 explained the ideas. This is what you type or read in this lab.

START IT

docker run

Starts your image as a container. Flags set config, ports, files.

SEE WHAT IT DOES

Python logging

One console line per request. Read it with docker logs.

TEST IT FROM OUTSIDE

curl or a script

Send real requests to the running container, from outside.

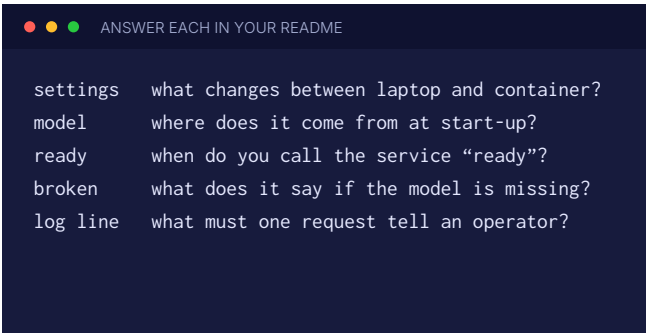
Any tool that gives the same evidence is fine; the evidence matters, not the brand.

Questions your README must answer (1 of 2)

Five questions about running the service.

README QUESTIONS

Answer each question in your README: your choice, the option you rejected, and why. Several answers can be good.



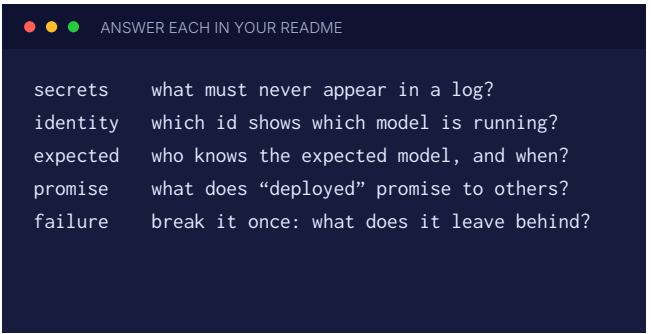
Examples and a command cheat sheet follow; they use a different project on purpose.

Questions your README must answer (2 of 2)

Five questions about evidence.

README QUESTIONS

Some answers depend on each other: what you log depends on who reads it.



Reviewers read your reasons; there is no single correct answer.

Rules that keep the evidence honest

Fixed for every team; everything else is your choice.

1 **Do not retrain**

The service only serves the Lab 4 model. Existing requests and responses keep working; you may add fields, not remove them.

2 **Check the running container**

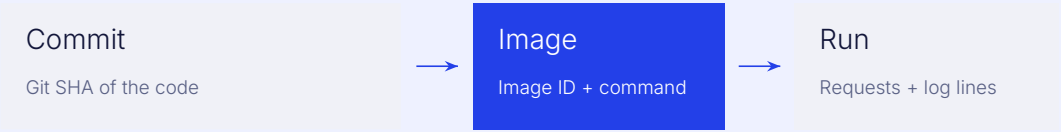
Send requests to the container over the network. Do not call the app's code directly (for example TestClient).

3 **Keep evidence honest**

No secrets, personal data or test-period metrics in the image or logs. Publish the port to your machine only. Report only what you saw.

Link the result to what produced it

Your report must connect these three, in this order.



`A screenshot alone does not connect them.`

Keep raw logs your report can link to; at most three screenshots.

Example: what a log can answer

A toy /score service; which fields you need is your decision.

ILLUSTRATIVE EXAMPLE

Two requests, two lines. Without seeing either request, you can tell what happened, how long it took and which model answered.

stdout of the toy container

```
{"event":"scored","id":"r-17",
 "status":200,"ms":38,"model":"m-01"}
{"event":"rejected","id":"r-18",
 "status":422,"ms":2,"model":"m-01"}
```

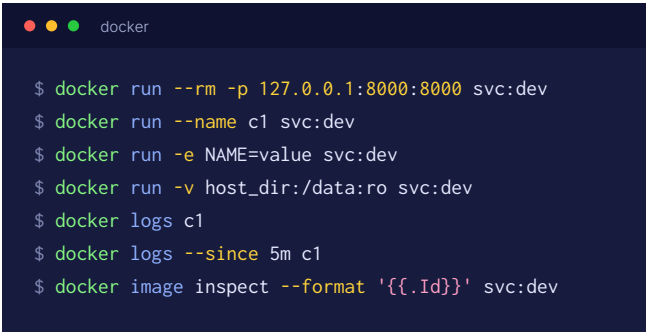
Different project on purpose. What to log, and what never to log, is your decision.

Cheat sheet: docker run flags

Values arrive at run time; you choose what to mount and what to publish.

RUN AND READ

Pass configuration with -e, publish ports to your own machine only, and read what the container says with docker logs.



Quoting differs in PowerShell and cmd; test the commands on your own machine.

Example: a check that can fail

A toy service again: each claim has an expected and an observed value.

ILLUSTRATIVE OUTPUT

One mismatch fails the whole check, and the message says which claim. Where “expected” comes from matters.

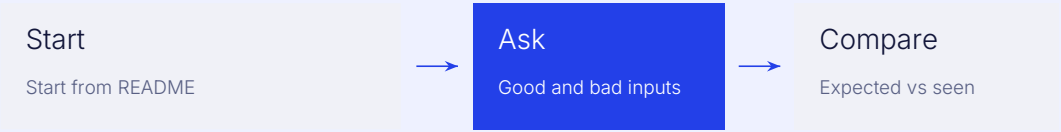
toy check output

```
claim      expected   observed   result
model      m-01       m-01       ok
revision   a1b2c3d    9f8e7d6    FAIL
ready      yes        yes        ok
exit status 1
```

Different project on purpose. Your claims, fields and exit behaviour are your design.

Prove it where it will run

Run the check against the container, not against your laptop code.



`A running container is not a verified deployment.`

Keep raw logs your report can link to; at most three screenshots.

Break it on purpose

A deployment is trusted only after you have seen it fail in a way you understand.

CHOOSE

Pick one failure

Choose something the service should refuse or survive.

OBSERVE

Cause it and look

Read the status code, the logs and your check output.

EXPLAIN

Find the cause

Could a stranger? If not, improve the logs.

Write down the failure you caused and what it left behind.

Review, blockers and handover

What a reviewer checks and what an honest record contains.

REVIEW

Follow the README

Use only the README. Change one thing; does the evidence show it?

BLOCKED

If you are stuck

Write the failing step, the exact message and who acts next.

HANDOVER

Merge, then Lab 6

Merge after review and checks. Lab 6 puts these checks in CI.

Local runs are not remote CI; keep them as separate claims.

m3 and your individual report

One team checkpoint for Labs 5–6; one personal report.

TEAM

Tag m3

After Labs 5 and 6 are reviewed, submit the link to an annotated tag m3.

PERSONAL

reports/week-3.md

Your own work, one decision, one blocker, any AI help.

RULES

On time, honest

Personal main, no PR or CI. Sunday 23:59. 10% of the course.

Incomplete work is reported as incomplete; never invent a result or a passing check.

EXIT CHECK

Could a stranger tell which model and code are answering?

Next, Lab 6: put these checks in CI, and make them able to fail.

Where the examples come from

Condensed from public documentation; read the originals for every option.

REFERENCES

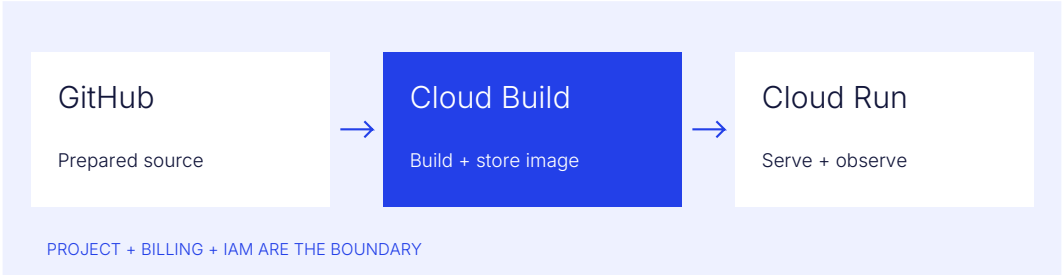
Commands are shortened and regrouped for this lab. Check the docs when a flag is unclear.



Illustrative examples; run them in your own project before you rely on them.

From GitHub to Cloud Run

Follow one deployment path step by step, using screenshots of a working demo.



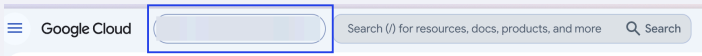
Same API and model; a managed runtime, not a new ML requirement.

Use the project and permissions your instructor gives your team.

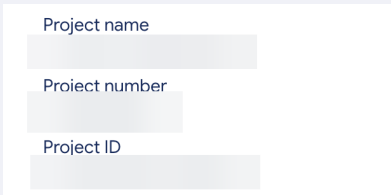
Select the project before deploying

Project selector → New project (if needed) → approved billing.

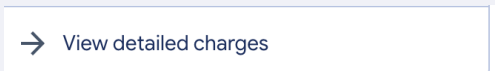
PROJECT SELECTOR



PROJECT NAME / NUMBER / ID



BILLING → VIEW DETAILED CHARGES



- Check the project scope; link only an instructor-approved billing account.
- Creation and payment screens are not supplied; this capture starts after creation.

Know who can change this project

IAM & Admin → IAM → inspect principals, roles and scope.

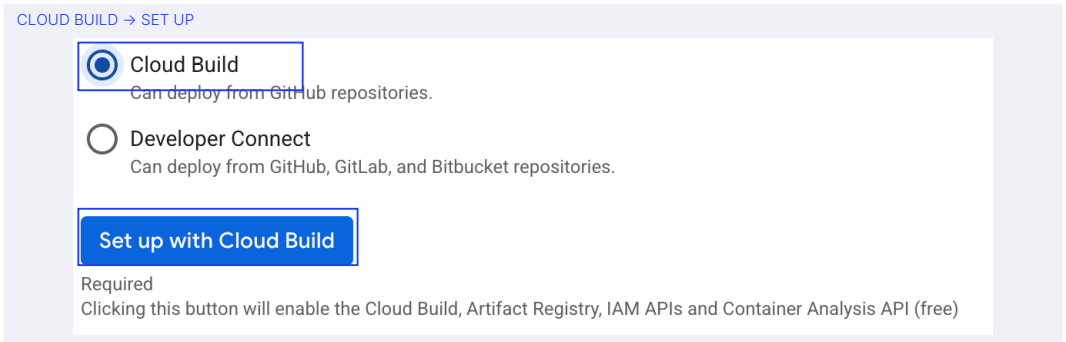
PRINCIPALS + ROLES



- Principal = who; role = allowed actions; project = where those actions apply.
- The capture shows instructor admin access, not a student role template.

Start with a repository, not a function

Cloud Run → Services → create service / connect repository.

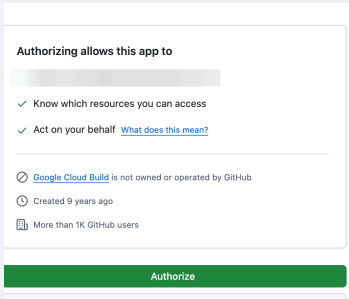


- Choose repository deployment, then Cloud Build → Set up with Cloud Build.
- Review the API-enablement prompt; builds and image storage can incur costs.

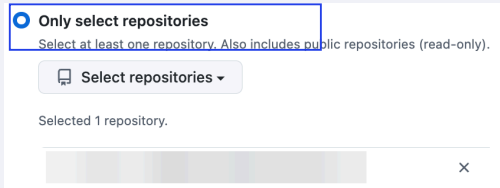
Authorize only the demo repository

Authenticate GitHub → review access → install the Cloud Build app.

1 / AUTHORIZE



2 / ONLY SELECT REPOSITORIES



3 / INSTALL

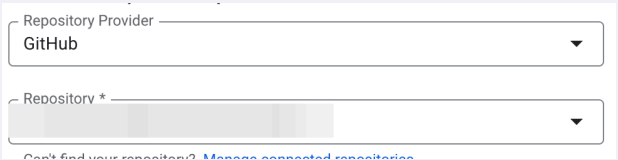


- Check the active GitHub account; choose Only select repositories.
- Grant access to the private demo copy only, not every repository.

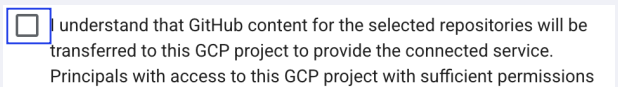
Check what Cloud Build will deploy

Choose the prepared repository → review consent → Next.

SOURCE REPOSITORY



REVIEW THE TRANSFER CONSENT

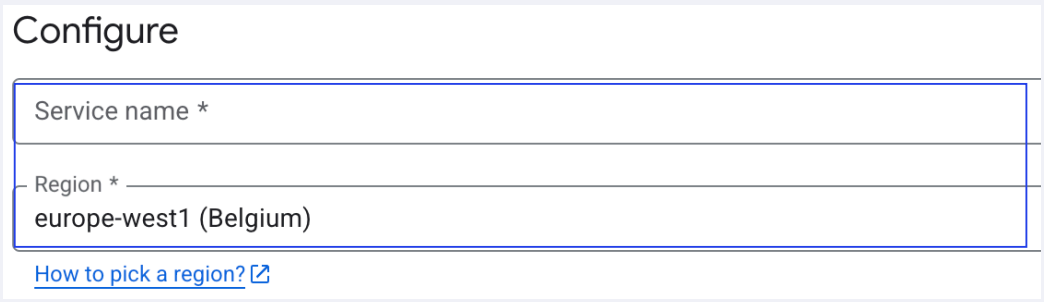


- In Build Configuration, use the root Dockerfile and its final service stage.
- Review branch / trigger settings: connected pushes can rebuild and redeploy.

Name the service and choose its region

Configure → service name → region.

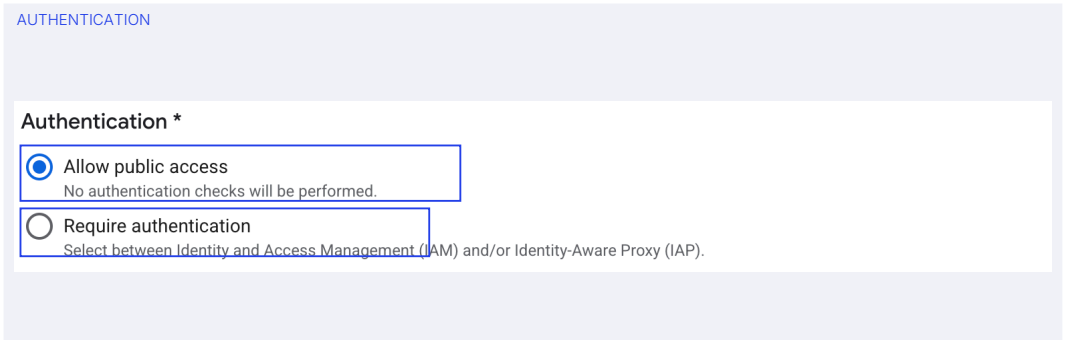
SERVICE NAME + REGION



- Use a clear demo name; this walkthrough uses europe-west1 (Belgium).
- Check the project and region before Create; do not copy a hidden project ID.

Public access is a deliberate exception

Authentication → choose who may invoke the service.

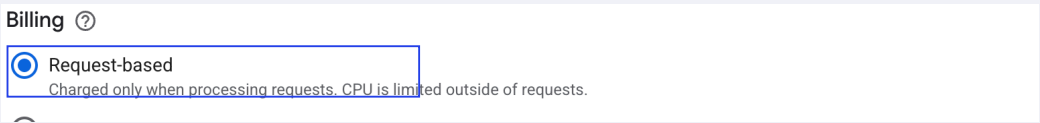


- This capture uses public access so the instructor can call Postman without a token.
- Prefer Require authentication for retained services; then clients need an ID token.

Scale to zero; do not assume zero cost

Billing → Request-based. Service scaling → Auto scaling.

REQUEST-BASED BILLING



MIN 0 / MAX 1



- Use min 0 / max 1 for a small demo; cold starts are acceptable.
- Recheck saved scaling after redeploy. Budget alerts are not spending caps.

Read the failure before changing settings

Revision History → failed revision → error details / logs.

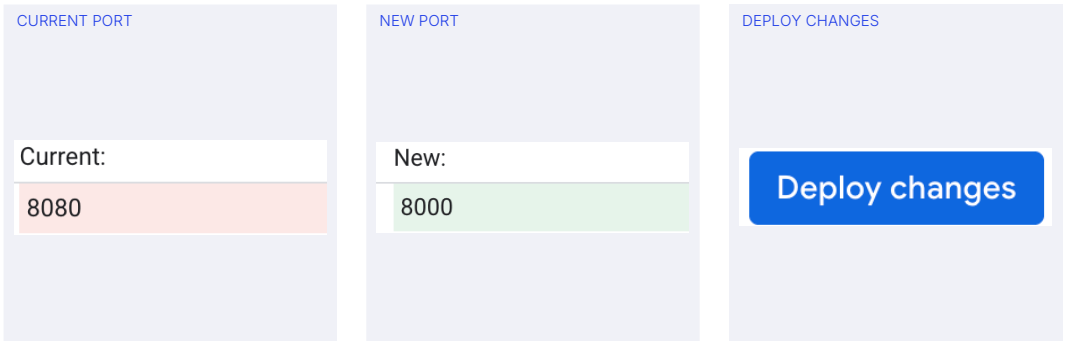
ACTUAL STARTUP FAILURE

> The user-provided container failed to start and listen on the port defined provided by the PORT=8080 environment variable within the allocated timeout. This can happen when the container port is misconfigured or if the timeout is too short. The health check timeout can be extended. Logs for this revision might contain more information. Logs URL: Open Cloud Logging For more troubleshooting guidance, see https://docs.cloud.google.com/run/docs/troubleshooting#container-failed-to-start

- The error names PORT=8080; the demo image starts Uvicorn on port 8000.
- Check the process and logs: a port error is not automatically a slow model.

Match the configured port to the process

Containers → container port → 8000 → review → Deploy changes.

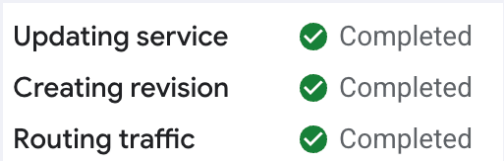


- This image uses a fixed 8000; the Cloud Run port setting must match it.
- Review traffic migration. A new deployment creates a new revision.

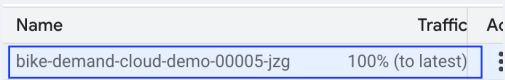
A ready revision is only the first check

Wait for deployment completion → inspect revision and traffic.

ALL THREE DEPLOYMENT STEPS



REVISION + TRAFFIC



- Confirm the intended revision receives traffic, then copy the service URL.
- Green deployment status does not prove model quality or all API tests.

Ask the live service which model it loaded

Postman → run 01 / Health before prediction.

GET {{base_url}}/health



OBSERVED RESPONSE: 200



- The response reports status, model_version and the model file SHA256.
- Compare with the expected artifact record; matching API fields alone is not enough.

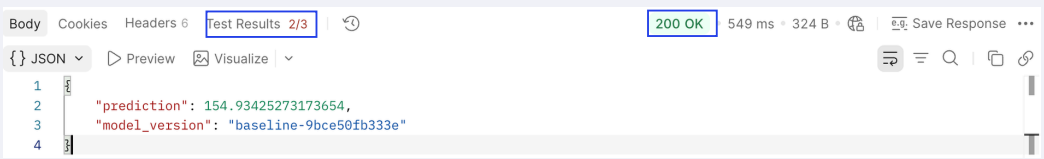
A prediction can succeed while a test fails

Postman → 02 / Valid prediction → inspect response and Test Results.

POST {{base_url}}/predict



OBSERVED: HTTP 200, TESTS 2/3



- The API returned a prediction and model_version; the capture reports 2/3 tests.
- Open Test Results and diagnose the failed assertion before claiming all checks pass.

Bad input should fail at the boundary

Postman → 03 / Invalid hour: hr = 24.

HTTP 422 / TESTS 2 OF 2

Test Results 2/2 | 422 Unprocessable Entity (WebDAV) (RFC 4918)

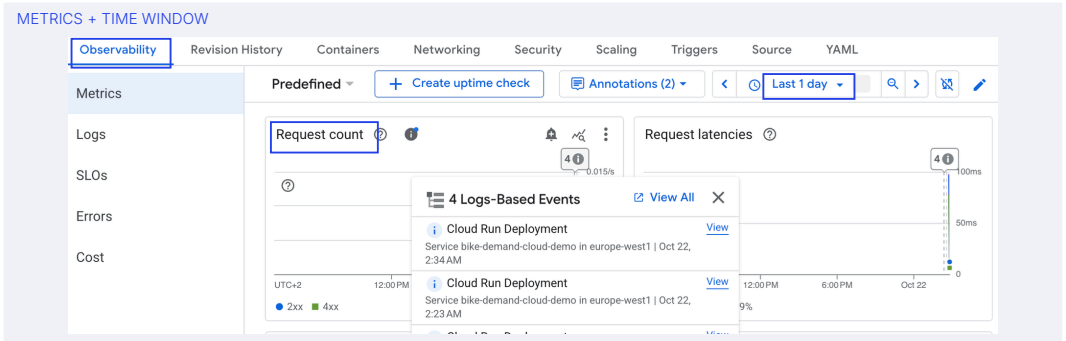
DETAIL: THE HOUR RANGE


```
"detail": [
    {
        "type": "value_error",
        "loc": [
            "body"
        ],
        "msg": "Value error, hr: require integers within [0, 23]",
```


- Read detail: the hour must be an integer in [0, 23].
- Change one field from the valid control; a 422 is expected rejection, not an outage.

Find the request signals in the console

Service → Observability → Metrics.

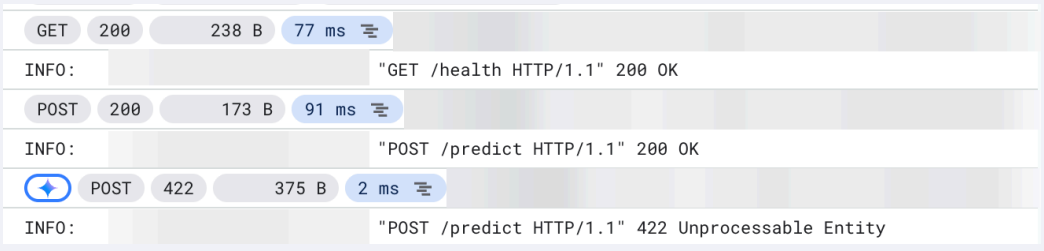


- Look for request count, status classes and latency; choose a matching time window.
- A few demo requests are not a load test or an SLO measurement.

Trace the requests in Logs

Service → Observability → Logs → choose the time window.

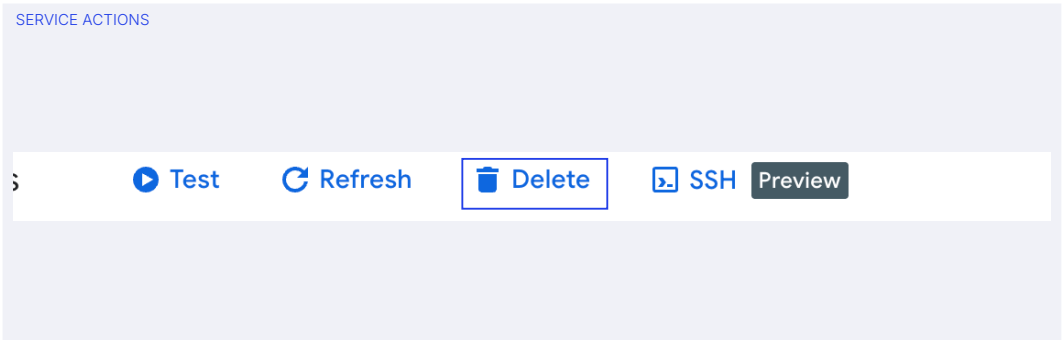
METHOD / STATUS / LATENCY / ROUTE



- Match GET /health, POST /predict and the expected 422 to their log entries.
- These are platform/access logs; they do not replace Lab 5 structured app events.

Finish by reducing exposure and cost

Service toolbar → Delete if no longer needed; otherwise secure and review.



- If retained: require authentication, verify min 0 / max 1 and disable unused build triggers.
- If finished: delete the service; review stored images, triggers and billing separately.